

Задание 10.1

Сущность	Обоснование
products	Большое кол-во операций чтения с фильтрацией и просмотром карточки товаров. Можно использовать несколько таблиц под популярные запросы (по категории, по бренду, топы...)
orders	Большое количество операций записи (создание заказа, обновление статусов). Удобно запрашивать по order_id (детали заказа) и иметь отдельную таблицу для истории заказов пользователя .
carts	Очень частые изменения (добавление/удаление позиций, изменение количества). Доступ в основном по ключу (user_id/cart_id), что идеально для Cassandra. Можно использовать TTL для автоочистки брошенных корзин/позиций.
order_items	Все позиции одного заказа хранятся в одной партиции по order_id, поэтому читаются одним запросом. Хорошо масштабируется и даёт предсказуемую задержку.
user_sessions	Доступ по ключу (session_id) позволяет быстро выполнять чтение/запись, TTL позволяет автоматически удалять истекшие сессии.

Задание 10.2

Products

product_id — единственный ключ доступа к карточке. Равномерно распределяет данные по узлам кластера

получить карточку конкретного товара

```
CREATE TABLE shop.product_by_id (  
  product_id    text,  
  title         text,  
  brand        text,  
  category     text,  
  price        decimal,  
  updated_at   timestamp,  
  attributes   text,  
  PRIMARY KEY ((product_id))  
);
```

Товары по категории, отсортированные по цене

Partition key — category — все товары одной категории хранятся на одном узле. Clustering key — price, product_id — обеспечивает нативную сортировку по возрастанию цены; product_id добавлен для уникальности строки.

```
CREATE TABLE shop.products_by_category (  
    product_id    text,  
    category      text,  
    price         decimal,  
    name          text,  
    attributes    text,  
    PRIMARY KEY ((category), price, product_id)  
) WITH CLUSTERING ORDER BY (price ASC, product_id ASC);
```

Товары по бренду, отсортированные по дате

Составной partition key — (brand, day) — предотвращает неограниченный рост партии ; Clustering key — (updated_at DESC, product_id DESC) — возвращает товары отсортированные по дате обновления без ORDER BY на стороне клиента.

```
CREATE TABLE shop.products_by_brand_day (  
    brand         text,  
    day           date,  
    updated_at    timestamp,  
    product_id    text,  
    title         text,  
    price         decimal,  
    category      text,  
    PRIMARY KEY ((brand, day), updated_at, product_id)  
) WITH CLUSTERING ORDER BY (updated_at DESC, product_id DESC);
```

Orders

Заказ по order_id

Partition key — order_id (UUID) — каждый заказ изолирован в своей партии

```
CREATE TABLE shop.order_by_id (  
    order_id      uuid,  
    user_id       text,  
    status        text,  
    created_at    timestamp,  
    updated_at    timestamp,  
    total_amount  decimal,  
    currency      text,  
    shipping_addr text,  
    payment_ref   text,  
    PRIMARY KEY ((order_id))  
);
```

История заказов пользователя

Составной partition key — (user_id, day) — не даёт партии расти бесконечно. Clustering key — (created_at DESC, order_id) — выдаёт заказы в хронологическом убывающем порядке без сортировки на клиенте

```
CREATE TABLE shop.orders_by_user_day (  
    user_id      text,  
    day          date,  
    created_at   timestamp,  
    order_id     uuid,  
    status       text,  
    total_amount decimal,  
    currency     text,  
    PRIMARY KEY ((user_id, day), created_at, order_id)  
) WITH CLUSTERING ORDER BY (created_at DESC, order_id DESC);
```

Carts

TTL задаётся на двух уровнях: `default_time_to_live` на уровне таблицы — применяется ко всем записям автоматически; `USING TTL` на уровне конкретной `INSERT/UPDATE` — переопределяет значение таблицы для этой записи

Активная корзина пользователя

Partition key — `user_id` — у пользователя всегда одна активная корзина; доступ по ключу пользователя — самый типичный запрос

```
CREATE TABLE shop.active_cart_by_user ( user_id      text, cart_id
uuid, updated_at  timestamp, currency    text, total_amount
decimal, expires_at  timestamp, PRIMARY KEY ((user_id)) )
WITH default_time_to_live = 2592000;
```

```
INSERT INTO shop.active_cart_by_user (user_id, cart_id, updated_at, currency,
total_amount, expires_at) VALUES
( 'user_123', uuid(), toTimestamp(now()), 'RUB', 0, toTimestamp(now()) )
USING TTL 604800;
```

Корзина по сессии

Partition key — `session_id` — корзина анонимного пользователя привязана к сессии и читается/пишется строго по её идентификатору. TTL позволяет Cassandra автоматически удалять устаревшие записи

```
CREATE TABLE shop.active_cart_by_session (
    session_id    text,
    cart_id       uuid,
    updated_at    timestamp,
    expires_at    timestamp,
    PRIMARY KEY ((session_id))
) WITH default_time_to_live = 2592000;
```

```
INSERT INTO shop.active_cart_by_session
(session_id, cart_id, updated_at, expires_at)
VALUES (
    'xyz',
    uuid(),
    toTimestamp(now()),
    toTimestamp(now())
) USING TTL 7200;
```

Order Items

Позиции заказа по order_id

Partition key — order_id — все позиции одного заказа хранятся вместе; SELECT по order_id возвращает их одним запросом к одному узлу. Clustering key — line_no ASC — естественная нумерация строк заказа;

```
CREATE TABLE shop.order_items_by_order (  
    order_id    uuid,  
    line_no     int,  
    product_id  text,  
    title_snapshot text,  
    qty         int,  
    price_snapshot decimal,  
    currency    text,  
    PRIMARY KEY ((order_id), line_no)  
) WITH CLUSTERING ORDER BY (line_no ASC);
```

User Sessions

сессия по идентификатору

Partition key — session_id — сессии идентифицируются по уникальному токену, который приходит в каждом запросе; TTL на уровне таблицы устраняет необходимость явной очистки: Cassandra удалит истёкшие сессии.

```
CREATE TABLE shop.session_by_id (  
    session_id    text,  
    user_id       text,  
    created_at    timestamp,  
    last_seen     timestamp,  
    ip            text,  
    user_agent    text,  
    payload_json  text,  
    PRIMARY KEY ((session_id))  
) WITH default_time_to_live = 1209600;
```

Задание 10.3

Сущность	Стратегия	Комментарий
products	Read Repair	Много чтений, мало изменений. Удобно исправлять расхождения на чтении.
orders	Anti-Entropy Repair	Предотвращаем накопление длительных расхождений.
carts	Hinted Handoff	Высокая нагрузка. Низкая задержка важнее чем актуальность данных
order_items	Anti-Entropy Repair	Предотвращаем накопление длительных расхождений.
user_sessions	Hinted Handoff	Важна скорость. Долгие расхождения не страшны, потому что сессия скоро истечёт